

Documentation Utilisateur

GL10
DOC



Présentation

Cette documentation est disponible en ligne avec des exemples interactifs [ici](#)

Compilateur Decac

Fonctionnalités

Instructions

- Variables
- Conditions
- while
- readInt(), readFloat()
- print(), println(), printx(), printlnx()
- return

Expressions

- Comparaisons logiques
- Opérations arithmétiques
- Opérateur unaire
- Null

Classes

- Classes
- Héritage
- Objet
- Conversion
- Méthodes Assembleur

Erreurs

- Erreurs à la compilation
- Erreurs à l'exécution

Extensions

ASTUCE

Activer toutes les extensions avec l'option `-f`

- Tableaux `-farray`
- Math
- Messages d'erreurs `-ffancy-errors`
- String `-fconcat-string`
- User IO
- Paint
- Assertions `-fassert`

Limitations

Rien à préciser.

Arithmétique

Plus

Plus est un terme qui exprime l'addition de deux quantités. Si a et b sont deux nombres, alors $a + b$ signifie a *plus* b et représente la somme des deux nombres.

Condition L'expression de l'**addition** (*plus*) n'est valable que dans les cas où le type des deux opérandes est soit un entier, soit un décimal.

Exemple

Exemple

```
print(1 + 1);
```

```
>> 2
```

Erreurs possibles

Arithmetic operations are only applicable to int and float types

Moins

Moins (*minus*) est un terme qui exprime la soustraction de deux quantités. Si a et b sont deux nombres, alors $a - b$ signifie **a moins b** et représente la différence entre les deux nombres.

Condition

L'expression de la **soustraction** est valide uniquement dans les cas où le type des deux opérandes est soit un entier, soit un décimal.

Exemple

```
print(5 - 2);
```

```
>> 3
```

Erreurs possibles

Arithmetic operations are only applicable to int and float types

Multiplication

La **multiplication** (*mult*) est une opération qui exprime la répétition d'une quantité un certain nombre de fois. Si `a` et `b` sont deux nombres, alors `a * b` signifie *a multiplié par b* et représente leur produit.

Condition

L'expression de la **multiplication** est valide uniquement dans les cas où le type des deux opérandes est soit un entier, soit un décimal.

Exemple

```
print(3 * 4);
```

```
>> 12
```

Erreurs possibles

Arithmetic operations are only applicable to int and float types

Divison

La **division** (*divide*) est une opération qui consiste à répartir une quantité en parts égales. Si `a` et `b` sont deux nombres, alors `a / b` signifie **a divisé par b** et représente le quotient de `a` par `b`.

Condition

L'expression de la **division** est valide uniquement dans les cas où le type des deux opérandes est soit un entier, soit un décimal.

Exemple

```
print(10 / 2);  
  
>> 5
```

Erreurs possibles

Arithmetic operations are only applicable to int and float types

Modulo

Le **modulo** (*mod*) est une opération mathématique donnant le reste de la division d'une variable par un nombre donné. Si a et b sont deux nombres entier, alors `a % b` signifie **a modulo b** et représente le reste de la division de a par b.

Condition

L'expression du **modulo** est valide uniquement entre deux entiers.

Exemple

```
print(10 % 2);  
  
>> 0
```

Erreurs possibles

Arithmetic operations are only applicable to int and float types

Moins unaire

Le **moins unaire** (*unary minus*) est une opération unitaire (*unaire*) qui s'applique à un seul opérande numérique. Elle sert à inverser le signe d'un nombre. Si `a` est un entier, alors `-a` signifie *l'opposé de a*.

Condition

L'expression du **moins unaire** s'applique uniquement sur un entier ou un flottant.

Exemple

```
print(--10);
```

```
>> 10
```

Erreurs possibles

Incompatible type for unary minus : expected float or int, got X

Comparaison logique

ET logique

L'opérateur ET logique `&&` (conjonction logique) renvoie `vrai` si et seulement si ses deux opérandes sont `vrai` ou équivalents à `vrai`.

Syntaxe

```
expr1 && expr2
```

Exemple

```
print(1 == 1 && 2 == 2) // true  
print(1 == 1 && 2 == 3) // false
```

Table de vérité

A	B	A && B
V	V	V
V	F	F
F	V	F
F	F	F

Comportement interne

Le ET logique est paresseux.

Si `expr1` est faux, `expr2` ne sera pas évalué et la valeur retournée par le ET logique sera faux

Erreurs possibles

All Operands should be boolean

OU logique

L'opérateur OU logique `||` (disjonction logique) renvoie `vrai` si et seulement si une des deux opérandes est `vrai` ou équivalents à `vrai`.

Syntaxe

`expr1 || expr2`

Exemple

```
print(1 == 1 || 2 == 2) // true
print(1 == 1 || 2 == 3) // true
print(1 == 2 || 2 == 3) // false
```

Table de vérité

A	B	A && B
V	V	V
V	F	V
F	V	V
F	F	F

Comportement interne

Le OU logique est paresseux.

Si `expr1` est vrai, `expr2` ne sera pas évalué et la valeur retournée par le ET logique sera `vrai`

Erreurs possibles

All Operands should be boolean

Inférieur strict

L'opérateur Inférieur strict `<` renvoie `vrai` si et seulement si l'opérande gauche est numériquement plus petite que l'opérande droit.

Syntaxe

```
expr1 < expr2
```

Exemple

```
print(1 < 2) // true
print(1 < 1) // false
print(2 < 1) // false
```

Erreurs possibles

Cannot compare operands of different types
Compared operands aren't supported

Inférieur ou égal

L'opérateur Inférieur ou égal `<=` renvoie `vrai` si et seulement si l'opérande gauche est numériquement plus petite ou égale que l'opérande droit.

Syntaxe

```
expr1 <= expr2
```

Exemple

```
print(1 <= 2) // true  
print(1 <= 1) // true  
print(2 <= 1) // false
```

Erreurs possibles

Cannot compare operands of different types
Compared operands aren't supported

Égale

L'opérateur Égale `==` renvoie `vrai` si et seulement si l'opérande gauche est numériquement égale à l'opérande droit.

Syntaxe

```
expr1 == expr2
```

Exemple

```
print(1 == 2) // false  
print(1 == 1) // true  
print(2 == 1) // false
```

Erreurs possibles

Cannot compare operands of different types
Compared operands aren't supported

Différent

L'opérateur Différent `!=` renvoie `vrai` si et seulement si l'opérande gauche est numériquement différent à l'opérande droit.

Syntaxe

```
expr1 != expr2
```

Exemple

```
print(1 != 2) // true
print(1 != 1) // false
print(2 != 1) // true
```

Erreurs possibles

- Cannot compare operands of different types
- Compared operands aren't supported

Supérieur ou égal

L'opérateur Supérieur ou égale `>=` renvoie `vrai` si et seulement si l'opérande gauche est numériquement plus grande ou égale que l'opérande droit.

Syntaxe

```
expr1 >= expr2
```

Exemple

```
print(1 >= 2) // false
print(1 >= 1) // true
print(2 >= 1) // true
```

Erreurs possibles

- Cannot compare operands of different types
- Compared operands aren't supported

Supérieur strict

L'opérateur Supérieur strict `>` renvoie `vrai` si et seulement si l'opérande gauche est numériquement plus grande que l'opérande droit.

Syntaxe

```
expr1 > expr2
```

Exemple

```
print(1 > 2) // false
print(1 > 1) // false
print(2 > 1) // true
```

Erreurs possibles

- Cannot compare operands of different types
- Compared operands aren't supported

Null

Le mot clé `null` permet d'initialiser une variable de classe sans instancier une instance de ladite classe.

Il est possible de comparer une variable avec `null` ou ses attributs (non primitif) pour vérifier s'ils ont été initialisés.

Exemple

```
class A{}

{
    A a = null
    if (a == null) {
        ...
    }
}
```

❗ REMARQUE

Null ne peut être utilisé pour comparer des types primitifs tels que les valeurs numériques.

Erreurs liées

Memory error: Dereferencing a null value

Cannot compare operands of different types

Opérateur Unaire

L'**opérateur unaire/inversion** (ou *not*) est une opération unaire qui s'applique à une valeur de type booléen (`True` ou `False`) et retourne sa valeur opposée. Elle est souvent représentée par l'opérateur `not`.

Condition

L'expression de l'**inversion** est valide uniquement si le **type_unary_op** respecte les règles suivantes :

type_unary_op : $\text{Operator} \times \text{Type} \rightarrow \text{Type}$

```
type_unary_op(not, boolean)  $\triangleq$  boolean
```

L'**inversion** s'applique uniquement sur un boolean.

Exemple

```
boolean b = true;
print(!b);

>> false
```

Erreurs possibles

Condition must be of type boolean

Tableaux

Les tableaux sont des structures permettant de ranger des données de manière ordonnées.

💡 ASTUCE

Activer l'extension avec l'option `-farray`

Types autorisés

Un tableau peut contenir:

- Des `int`
- Des `float`
- Des `boolean`
- Des `string`
- Des objets
- Des tableaux

Initialisation

Ils s'initialisent avec un `new` et prennent une **taille** entière à l'initialisation.

```
{  
    int[] a = new int[5]; // tableau de 5 entiers  
}
```

Tableau de `int` ou `float` : Initialisation par défaut à 0 et 0.0
Tableau de `boolean` : Initialisation par défaut à `false`
Tableau d'objets ou de tableaux : Initialisation par défaut à `null`

Sélection et assignation

Pour récupérer ou assigner un élément du tableau, on appelle le tableau avec l'index de l'élément. Cet index doit être un entier. Il commence à 0, donc le 1er élément s'atteint via `tableau[0]`.

L'index ne peut pas être négatif.

```
{
    float[] b = new float[3]; // [0.0, 0.0, 0.0]
    float c;

    b[1] = 4.1; // [0.0, 4.1, 0.0]

    c = b[1]; // c = 4.1
}
```

Tableaux imbriqués

Comme dit précédemment, on peut construire des tableaux de tableaux, utiles notamment pour des matrices.

```
{
    int[][] a = new int[][3]; // [null, null, null]

    a[0] = new int[2]; // [[0,0], null, null]

    a[0][1] = 5; // [[0,5], null, null]
}
```

Accès à l'extérieur du tableau

En cas d'accès à un index égal ou supérieur à la taille du tableau, vous accéderez à une case mémoire non-allouée par l'initialisation du tableau. Le comportement est alors indéfini.

```
{  
    int[] a = new int[5];  
    int b;  
    b = a[5]; // b prend une valeur indéfinie  
}
```

Erreurs possibles

Logical error: Negative array index

Array size must be an integer

Index value type must be int

Array type can't be void

Cannot index non-array type

Arrays are not supported in this version of Deca

Assert

Cette extension du programme déca met à disposition du développeur la fonction `assert()`.

Cette fonction **fourni** une assertion ergonomique. Le message d'erreur permet de localiser efficacement l'assertion qui a échoué. Cette fonction **n'est pas** capable d'afficher un message personnalisé.

💡 ASTUCE

Activer l'extension avec l'option `-fassert`

Exemple

```
{  
    assert(booleanExpr);  
}
```

Argument

- `booleanExpr` doit obligatoirement être une expression de type `boolean`.
- Une erreur est levée si la condition est de type `int`, `float`, `string`, ou `void`.

Comportement

La fonction ne fait rien quand l'expression booléenne est évaluée à `true`. A l'inverse, le programme s'arrête avec une erreur affichant le message `Assertion failed at line: x` qui indique la ligne de l'assertion.

Erreurs possibles

Condition must be of type boolean Assertion failed at line: x

Messages d'erreurs

Affichage des messages d'erreurs contextuelles

Cette extension a pour but d'ajouter plus de précision dans le message d'erreur contextuelle, en affichant la ligne qui a causé l'erreur et en colorant en **rouge** la source exacte d'erreur.

💡 ASTUCE

Activer l'extension avec l'option `-ffancy-errors`

Utilisation

À fin de visualiser le message d'erreur coloré, il suffit de lancer le compilateur `decac` en lui passant en paramètre un fichier `.deca` contenant une erreur contextuelle.

```
decac src/test/deca/context/invalid/class/03AssignNullToInt.deca
```

Dans ce fichier de test, on initialise une variable **a** de type **int** à **null** ce qui n'est pas autorisé dans deca vu que **a** n'est pas un objet. Il s'agit donc d'une erreur contextuelle. La source d'erreur étant le **null**, le message d'erreur aura la ligne qui a causé l'erreur avec le mot **null** coloré en rouge comme ceci:

```
int a = null;
```

Math

L'extension *Math* permet à l'utilisateur d'utiliser une variété de méthodes mathématiques.

Utilisation

L'extension *Math* permet à l'utilisateur d'accéder à une large variété de méthodes mathématiques.

Ces méthodes sont accessibles en instanciant un objet de type *Math* après l'inclusion du fichier de classe, puis en appelant les méthodes depuis cet objet.

Exemple

```
#include "Math.decah"

class TestMath {
  Math m = new Math();
  // Exemple : m.sin(m.pi / 2);
}
```

Méthodes disponibles

Signature	Description
float abs(float a)	Retourne la valeur absolue de a.
float asin(float a)	Retourne l'arc sinus de a, en radians (intervalle [-π/2, π/2]).
float atan(float a)	Retourne l'arc tangente de a, en radians (intervalle [-π/2, π/2]).

Signature	Description
float ceil(float a)	Retourne le plus petit entier supérieur ou égal à a.
float cos(float a)	Retourne le cosinus trigonométrique de l'angle a (en radians).
float exp(float a)	Retourne l'exponentielle de a, soit e^a .
float floor(float a)	Retourne le plus grand entier inférieur ou égal à a.
float ln(float a)	Retourne le logarithme népérien de a.
float normalize(float a)	Ramène un angle en radians dans l'intervalle $[-\pi, \pi]$.
float pow(float a, float b)	Retourne a élevé à la puissance b (a^b).
float sin(float a)	Retourne le sinus trigonométrique de l'angle a (en radians).
float sqrt(float a)	Retourne la racine carrée de a.
float tan(float a)	Retourne la tangente trigonométrique de l'angle a (en radians).
float toDegrees(float rad)	Convertit un angle de radians en degrés.
float ulp(float f)	Retourne la plus petite différence représentable autour de f.
int fact(int n)	Retourne la factorielle de n.
int random()	Retourne un entier pseudo-aléatoire entre 1 et 4.

Constantes

Attribut	Valeur	Description
float pi	3.1415927	Valeur approchée de π .
float two_pi	2 * pi	Valeur de 2π .
float MIN_FLOAT	2 ⁻¹⁴⁹	Plus petite valeur positive non null

Point

Une classe `point` est fourni avec comme attribut x et y.

Exemple

```
#include "Point.decah"
{
    Point p1 = new Point();
    p1.x = 1;
    p1.y = 2;
}
```

Bezier

Exemple

```
#include "LinearBezier.decah"

class A {
    LinearBezier b = new LinearBezier();
}
```

La classe `LinearBezier` qui étend `Bezier` est fournis pour créer des courbes suivant l'algorithme de bézier.

Les classes basées sur `Bezier` ont des méthodes initialisables via la méthode `set` pour passer les points nécessaires à la courbe.

`LinearBezier` : `set(Point start, Point end)`

La méthode `getPointAtCurve(float x)` permet d'obtenir la coordonnée de la courbe à l'indice X compris entre 0 et 1.

ATTENTION

`QuadricBezier` et `CubicBezier` sont encore en cours de développement et seront rajoutés pour vendredi.

Paint

Cette extension du programme déca permet d'avoir une interface semi-graphique dans le terminal afin d'y réaliser des opérations de dessins à l'aide de méthodes géométrique.

Utilisation

Lancer le programme

```
#include "Paint.decah"
{
    Paint paint = new Paint();
    paint.start();
}
```

DANGER

Le programme doit être lancé avec une mémoire allouée assez grande `-t 999999`

En-tête

La barre en haut donne les indications d'utilisation avec :

- Un message de bienvenue.
- Des potentiels messages d'erreurs.
- La couleur sélectionnée.
- Le choix des couleurs possibles et la touche du clavier pour la sélectionner.
- Les indications des touches

Effacer le terrain de dessin

En appuyant sur `w` le tableau sera remis à 0.

Mode dessin

Appuyer sur **D** pour activer le mode dessin. Dans ce mode le clic de la souris dessine le pixel choisi.

Mode géométrie

Le mode géométrie permet de réaliser différentes opérations à partir de points.

Placement de points

Il est possible de placer jusqu'à 4 points. Pour se faire cliquer sur l'espace libre, puis une fois que vous êtes certains de la position, confirmer le placement en appuyant sur **ENTRER**.

Vous pouvez passer au point suivant.

Après avoir appliqué une opération géométrique, les points précédemment placés sont supprimés.

Ligne droite

Il vous faut entre 2 et 4 points pour utiliser cet outil.

Tracer une ligne droite entre des points en appuyant sur **G**, En appuyant sur **H** le dernier point et le premier point seront également reliés (fermant la forme).

ATTENTION

Pour cause d'arrondi de décimaux, il se peut que les lignes droites soient parfois imparfaites.

Remplissage

Après avoir placé les 4 points, appuyer sur **F** pour remplir la zone entre les 4 points.

Rectangle

Après avoir placé 2 points, appuyer sur **M** pour placer les 2 autres points complémentaires pour former un rectangle parfait.

Objet string

L'extension *string* autorise l'utilisateur à utiliser le type `string`. Ce type peut être affiché et être concaténé à une autre instance de `string`.

Utilisation

💡 ASTUCE

L'extension doit être activée avec l'option `-fstring-object` de Decac.

Les chaînes de caractères sont immutables et supportent toute séquence UTF-8 valide. Il est alors possible de placer une chaîne de caractères dans une variable, de la stocker dans un attribut et de la passer en paramètre d'une méthode.

```
class Greetings {
    void greet(string name) {
        println("Hello ", name);
    }
}

{
    string alice = "Alice";
    new Greetings().greet(alice); // Affiche "Hello, Alice"
    new Greetings().greet("Bob"); // Affiche "Hello, Bob"
}
```

L'opérateur `+` permet de concaténer deux chaînes de caractères, sous condition qu'aucune ne soit nulle. Par exemple :

```
{
    string alice = "Alice";
    string bob = "Bob";
```

```
println(alice + ", " + bob); // Affiche "Alice, Bob"
}
```

`null` peut être assigné à une variable de type `string`. Il s'agit par ailleurs de sa valeur par défaut dans les attributs d'un objet.

Il n'est pas possible d'afficher des chaînes de caractères nulles. Il peut par conséquent être utile de vérifier que la chaîne de caractères n'est pas nulle avant d'effectuer un traitement :

```
class Greetings {
    void greet(string name) {
        if (name == null) {
            println("Hello unknown!");
        } else {
            println("Hello ", name);
        }
    }
}

{
    new Greetings().greet(null); // Affiche "Hello unknown!"
}
```

Deux chaînes de caractères peuvent être comparés avec la méthode `equals` :

```
{
    string a = "abc";
    string b = "cba";
    if (!a.equals(b)) {
        println("different"); // Affiche "different"
    }
}
```

Enfin, il est possible de connaître la taille de la chaîne de caractères avec la méthode `length` :

```
{
    println("").length(); // Affiche 0
}
```

```
println("abc".length()); // Affiche 3  
}
```

Relation avec les objets

`string` est un type nullable sans être un type objet : il n'hérite donc pas de la classe `Object` et ne peut pas être étendu.

User I/O

Cette extension du programme déca met à disposition du développeur une extension qui gère les entrées et sorties sur la sortie standard du terminal executant le programme.

Cette extension **est** capable de réagir à un appui sur le clavier, un clic sur la souris, modifier le style de la sortie (couleurs).

Cette extension **n'est pas** capable de donner l'état des touches de contrôle (CTRL, SHIFT), la position de la souris (uniquement au clic).

Utilisation

L'extension fourni la classe `UserIO`, pour l'utiliser, il faut créer votre propre classe qui étend `UserIO` et configurer les différents `callback` en surchargeant les méthodes de la classe `UserIO`.

Pour lancer l'écoute de la sortie standard, il faut appeler la méthode `callbackOnInputExitable` en indiquant le code UTF8 de la clé de sortie.

```
class UserIOImplemented extends UserIO {
#### void onKeyInput(int key){
    print("Key n°", key, " have been pressed"); // Print l'identifiant
    UTF8 de key
    printUtf8(key) // Print l'interpretation UTF8
}

#### void onMouseEvent(MouseEvent me){
    if (me.isPressed) {
        this.setCursorAt(me.x, me.y);
        print("X");
        this.cursorHome();
        this.eraseLine();
    }
}
```

```
#### void onRPressed(){
    this.eraseFullScreen();
    this.cursorHome();
}

}
{
    UserIO u = new UserIOCallback();

    u.cursorHome();
    u.enableMouseInput();
    u.callbackOnInputExitable(113);
    println("Exit program");
}
```

Attributs

screenSize

Un objet de types `Coordinates` qui contient la taille du terminal de sortie.

ATTENTION

Certains terminaux ne supportent pas le changement de lignes et colonnes, cette taille est donc fictives et utiliser pour harmoniser les calculs.

me

Un objet de types `MouseEvent` qui contient les données du dernier clic souris.

Méthodes

int callbackOnInputExitable(int exitKey)

Méthode principale à appeler pour déclencher l'écoute sur la sortie standard.

Cette méthode est bloquante tant que la touche sur le clavier correspondant à `exitKey` en UTF8 n'a pas été appuyé.

MouseEvent `getLastMouseEvent()`

Getter de `me`

int `readUtf8()`

Renvoie un entier correspondant au caractère UTF8 lu sur l'entrée standard,

Renvoie -1 si rien n'est lu.

REMARQUE

N'est pas bloquant

Coordinates `getScreenSize()`

Renvoie un objet `Coordinates` avec la taille de l'écran.

void `enableMouseInput()`

Active le callback des clics sur la souris.

ATTENTION

Modifie le comportement du terminal même après exécution du programme.

void `eraseFullScreen()`

Efface tout l'écran.

void `eraseLine()`

Efface la ligne courante du curseur.

void `setCursorAt(int x, int y)`

Place le curseur à la position `x` et `y`.

void askCursorPosition()

Demande au terminal d'envoyer la position du curseur.

void cursorHome()

Déplace le curseur en haut à gauche de l'écran (0,0).

void printUtf8(int m)

Écrit dans la sortie standard l'interprétation UTF8 du décimal `m`

void sendCommandInt(int intCode)

Méthode utilitaire pour l'application des couleurs.

Envoie sur la sortie standard `ESC[intCode`

void setBlackBackground()

Change la couleur de fond pour les prochains print en noir.

void setRedBackground()

Change la couleur de fond pour les prochains print en rouge.

void setGreenBackground()

Change la couleur de fond pour les prochains print en vert.

void setYellowBackground()

Change la couleur de fond pour les prochains print en jaune.

void setBlueBackground()

Change la couleur de fond pour les prochains print en bleue.

void setMagentaBackground()

Change la couleur de fond pour les prochains print en magenta.

void setCyanBackground()

Change la couleur de fond pour les prochains print en cyan.

void setWhiteBackground()

Change la couleur de fond pour les prochains print en blanc.

void setDefaultBackground()

Change la couleur de fond pour les prochains print à la couleur par défaut.

void setScreenSize(int height)

Cette méthode change la taille de l'écran sur l'attribut `screenSize` et appelle la méthode `setScreenSizeASM`.

void setScreenSizeASM(int height)

Cette méthode tente de changer la taille de l'écran au niveau du terminal.

void resetStyle()

Cette méthode supprime tous les changements de couleurs et de style pour les prochains print.

Callback sur le clavier

void onKeyInput(int key)

Cette méthode est appelée à chaque fois qu'une touche (qui n'est pas une touche de contrôle) est appuyée sur le clavier.

`key` correspond à son identifiant décimal en UTF8.

void onMouseEvent(MouseEvent me)

Cette méthode est appelée à chaque fois que le bouton gauche de la souris est appuyé ou relâché.

`me` est un objet `MouseEvent` contenant les coordonnées du clic et si c'est un clic ou un relâchement.

void onArrowUp()

Cette méthode est appelé lorsque la flèche haut est pressée.

void onArrowDown()

Cette méthode est appelé lorsque la flèche basse est pressée.

void onArrowLeft()

Cette méthode est appelé lorsque la flèche gauche est pressée.

void onArrowRight()

Cette méthode est appelé lorsque la flèche droite est pressée.

void on_Pressed()

Cette méthode est appelé lorsque chaque lettre de l'alphabet est pressée.

Les 26 lettres sont implémentés

```
void onAPressed()  
void onBPressed()  
void onCPressed()  
...  
void onZPressed()
```

void onENTERPressed()

Cette méthode est appelé lorsque la touche ENTER est pressée.

Print

Print, Println

`Print` et `Println` sont des instructions permettant d'écrire dans la sortie standard du programme.

Ils peuvent prendre autant de paramètres que nécessaires.

Les paramètres doivent être de type `String`, `int`, `float` ou une variable du même type.

Erreurs possibles

Only string, int and float may be printed

Printx Printlnx

`Printx` et `Printlnx` sont respectivement identiques aux instructions `Print` et `Println` mais écrivent la valeur des décimaux en hexadecimal.

Conditions

If Else if Else

Les conditions permettent d'exécuter du code en fonction de valeur de variables.

Syntaxe

```
if (expr) {  
    code si vrai  
} else if (expr) {  
    code si seconde condition vrai  
}  
else {  
    code si faux  
}
```

Erreurs possibles

Condition must be of type boolean

Cannot compare operands of different types

Compared operands aren't supported

Operands compared should be float or int

All Operands should be boolean

ReadInt, ReadFloat

readInt()

L'instruction `readInt()` permet à l'utilisateur de saisir un entier durant l'exécution du code.

`readInt()` ne prend aucun paramètre et renvoi un entier.

Elle interrompt l'exécution du code jusqu'à ce que l'utilisateur appuie sur `ENTER`. Si la valeur saisie par l'utilisateur entre l'exécution sur code et sa validation n'est pas un entier valide, une erreur est levée et le programme s'arrête.

Exemple

```
int a = readInt();

int b;
b = readInt();
```

readFloat()

L'instruction `readFloat()` permet à l'utilisateur de saisir un décimal durant l'exécution du code.

`readFloat()` ne prend aucun paramètre et renvoi un décimal.

Elle interrompt l'exécution du code jusqu'à ce que l'utilisateur appuie sur `ENTER`. Si la valeur saisie par l'utilisateur entre l'exécution sur code et sa validation n'est pas un décimal valide, une erreur est levée et le programme s'arrête.

Exemple

```
float a = readFloat();
```

```
float b;
```

```
b = readFloat();
```

Return

return

L'instruction `return` permet de retourner une valeur à la fin de l'exécution d'une méthode.

Ce mot clé doit être utilisé uniquement si la fonction attend une valeur de retour (autre que void donc).

Exemple

```
class A {  
    int a(){  
        return 1;  
    }  
}
```

Erreurs possibles

Cannot return a void value

Variables

Déclaration de variable

Les variables permettent de stocker une information et de la modifier ultérieurement.

Les variables **doivent** être déclarées avant de pouvoir être utilisées. Dans le cas contraire une erreur est levée à la compilation.

Une variable peut être déclarée sans être initialisée, tout comme elle peut être initialisée.

Syntaxe de déclaration de variable

```
type name [= valeur]
```

Types disponibles

- int
- float
- boolean
- classes
- string

Le nom

Le nom peut être une suite de caractère et de chiffres ainsi que les caractères spéciaux suivants : _ \$ Le nom ne PEUT PAS commencer par un caractère numérique.

Valeur

La valeur est optionnelle, elle permet d'initialiser la variable, la valeur doit être du même type que le type de la variable.

Le compilateur supporte le fait d'initialiser une variable float avec un entier (int) et réalise la conversion à la compilation.

Erreurs possibles

Variable X is already declared

Modification des variables

Une variable peut être modifiée dans l'exécution du code, si elle est dans la portée du code via la syntaxe suivante :

```
int a;  
a = 2;  
a = 3;
```

Erreurs possibles

Incompatible types for assignment: expected X, got Y

Field X is already declared

While

L'instruction **while** permet de répéter un bloc d'instructions tant qu'une condition est vraie. Elle est utilisée pour exprimer des boucles à condition d'entrée, c'est-à-dire que la condition est vérifiée avant chaque itération.

Exemple

```
{
    while(condition){
        bloc_instructions
    }
}
```

- **condition** : une expression de type **boolean**.
- **bloc_instructions** : peut contenir une ou plusieurs instructions.
- **Comportement** :
 - Tant que la **condition** est évalué à **true**, alors bloc_instructions est exécuté.
 - Sinon, si la condition est évalué à **false**, la boucle est terminée.
- **Types autorisés** :
 - **condition** doit obligatoirement être une expression de type **boolean**.
 - Une erreur est levée si la condition est de type **int**, **float**, **string**, ou **void**.

Erreurs possibles

Condition must be of type boolean

Cannot compare operands of different types

Compared operands aren't supported

Operands compared should be float or int

All Operands should be boolean

Classes

Déclaration des classes

Les classes se déclarent via la syntaxe suivante :

```
class Foo {  
    // attributs et méthodes de la classe  
}  
{  
    // main ...  
}
```

Les classes doivent être déclarées avant le corps `main` du programme.

Une classe est composée d'attributs et de méthodes.

Erreurs possibles

Field cannot have type void

Field X is already declared

Class already exist

Instanciation

Pour créer une nouvelle instance d'une classe et la stocker dans une variable, il faut déclarer la variable avec le type correspondant à la classe, où une classe parente.

Et l'initialiser en appelant la classe précédée du mot clé `new`.

À l'instantiation d'une classe, les attributs sont initialisés avec leur expression [voir valeur par défaut](#).

Exemple

```
class Foo {  
    Foo a = new Foo();  
}
```

Erreurs possibles

X does not refer to a value/type.

Attributs

Les attributs sont déclaré sous la syntaxe suivante :

```
[protected] {type} ({name} [ = {value}])*
```

Un attribut est publique par défaut. Il est donc lisible et écrivable librement par tout code ayant accès à la classe.

Pour les attributs `protected` voir [Héritage#protected](#).

Pour lire un attribut, il faut écrire l'identifiant de l'instance de la classe puis le nom de l'attribut séparé par un point `.`

Exemple

```
class A {  
    int foo = 2;  
}  
A a = new A();  
println(a.foo); // 2
```

Pour modifier la valeur d'un attribut il faut ajouter l'opérateur égal `=` suivi de la nouvelle valeur.

Exemple

```
class A {  
    int foo = 2;  
}  
A a = new A();  
a.foo = 3;  
println(a.foo); // 3
```

Erreurs possibles

Incompatible types for assignment: expected X, got Y

Field X is already declared

Object X does not have an attribute Y

Valeur par défaut

Les valeurs par défaut sont :

- entier : 0
- décimaux : 0.0
- booléen : false
- classe : null

Il est possible de changer la valeur par défaut en ajoutant une expression après la déclaration, séparé d'un `=`.

Le comportement des valeurs par défaut permet d'utiliser un autre attribut dans une déclaration.

Cependant, si l'attribut est situé sur une ligne après la déclaration actuelle, la valeur de l'attribut sera équivalent à 0, car il est déclaré, mais en encore initialisé.

Si l'attribut est sur une ligne précédant la déclaration actuelle, la valeur utilisée sera sa valeur après initialisation.

Exemple

```
class Foo{  
    int x = y*y;
```

```
int y = 5;
int z = y*y;
}
```

Ce code ne déclenche pas d'erreur, cependant `x` et `z` auront une valeur différente :

```
{
    Foo foo = new Foo();
    println(foo.y); // 5
    println(foo.x); // 0
    println(foo.z); // 25
}
```

Ce comportement est dû au fait qu'au moment de l'initialisation de `x`, `y` vaut 0, car il est connu, mais pas encore initialisé. Au moment de l'initialisation de `z`, `y` a été initialisé à 5, donc `z` vaut 25.

Accès à l'attribut

Pour accéder aux attributs d'une classe, il faut indiquer le nom de classe suivi de l'attribut séparé par un point.

Si l'attribut est lui-même une classe avec d'autres attributs, alors cela peut être chaîné pour accéder à l'attribut final.

Exemple

```
Foo foo = new Foo();
Bar bar = new Bar();
int a = foo.bar.x;
```

Erreurs possibles

Unable to select a field of a non-class type

Object X does not have an attribute Y

Memory error: Dereferencing a null value

Méthodes

Les méthodes sont des fonctions qui peuvent modifier ou non les attributs d'une classe, réaliser des calculs et renvoyer le résultat. L'objectif est de fournir un moyen de réutiliser du code sans le dupliquer.

Syntaxe

```
class A {  
    {type_retour} {nom} () {  
        //...instructions de la méthode  
    }  
}
```

Pour manipuler les attributs dans une méthode, il suffit de mentionner le nom de l'attribut, pour lever toute ambiguïté si un paramètre porte le même nom, on utilisera le mot clé `this`.

Les méthodes peuvent retourner une variable grâce au mot clé `return` à utiliser si la méthode possède un type de retour.

Exemple

```
class Foo {  
    int a = 2;  
  
    int inca(int a){  
        this.a = this.a + a;  
        return this.a;  
    }  
}
```

Erreurs possibles

Param X is already declared

Method X is overridden with different number of parameters

Method X is overridden with a different return type

Method X is overridden using a Y parameter, while expecting type Z

Unable to call a method on the X primitive type

X does not refer to any existing method

Trying to use the X Y as a method

Does not match number of Method parameters of X, Y expected, but found Z

Parameter X does not match Method type of Y, expected A but found B

Cannot return a void value

Error while trying to access an attribute

Héritage

L'héritage permet à une classe d'avoir accès aux attributs et aux méthodes d'une autre classe.

Pour cela, il faut étendre la classe au moment de sa déclaration en utilisant le mot clé `extends`.

Exemple

```
class Foo {}  
class Bar extends Foo {}
```

Erreurs possibles

Superclass X is not a class

Field X cannot shadow a Y in the superclass

Attributs et Protected

Une classe enfant ne peut pas avoir des attributs propres qui ont le même nom que des attributs de la classe parente. De plus lors de l'instanciation des attributs l'utilisateur peut spécifier la visibilité de ceux-ci.

Il a donc le choix entre ne rien mettre, ce qui signifie une visibilité publique implicite, ou de rajouter le mot clé `protected`. Le mot clé `protected` empêche l'accès des attributs en dehors de la classe. Les classes enfants qui héritent des attributs `protected` de leurs parents, auront accès à ces attributs.

Exemple

```
class Parent {
    protected int x = 7;
}

class Enfant extends Parent {
    int y = x;
}
{
    println(new Enfant().y);
}
```

Redéfinition (Override)

Une classe enfant peut redéfinir une méthode déjà déclarée dans la classe parente pour redéfinir son comportement. Le nom de la méthode et les types des paramètres et de retour doivent être identiques.

Seul le type de retour peut être un type plus spécifique que celui de la méthode parente. Dans ce cas, appelé retour covariant, le type de retour de la méthode enfant doit être un sous-type du type de retour de la méthode parente.

Exemple

```
class Parent {
    int bar() {
        print(1);
    }

    int foo(int a) {
        print(a);
    }
}

class Enfant extends Parent {
    int foo(int a) {
        print(a + 1);
    }
}
{
    Enfant e = new Enfant();
```

```
print(enfant.bar()); // 1;  
print(enfant.foo(2)); // 3;  
}
```

Opérateur InstanceOf

L'opérateur `instanceof` permet de vérifier si le type effectivement instancié d'un objet est, ou est fils de, une classe donnée.

Erreurs possibles

Method X is overridden with different number of parameters

Method X is overridden with a different return type

Method X is overridden using a Y parameter, while expecting type Z

Protected field cannot be accessed outside of its class

Left operand of the instanceof needs to be an object

Right operand of the instanceof needs to be a class

Objet

Toutes les classes héritent implicitement de la classe `Object`.

Cette classe est instanciable.

La classe `Object` définit une méthode `equals` qui permet de comparer deux objets. Son implémentation par défaut compare les références en mémoire des objets, mais elle peut être redéfinie pour comparer les valeurs des attributs des objets.

Par exemple, si nous voulons créer une classe pour passer par référence un entier mutable, nous pouvons redéfinir la méthode `equals` pour comparer les valeurs de l'entier plutôt que les références.

```
class MutableInt {
    int value;

    boolean equals(Object other) {
        if (other instanceof MutableInt) {
            return this.value == ((MutableInt) (other)).value;
        }
        return false;
    }
}

{
    MutableInt x = new MutableInt();
    MutableInt y = new MutableInt();
    x.value = 5;
    y.value = x.value;
    if (x.equals(y)) { // Retourne vrai. Sans la redéfinition de equals, cette
condition serait fausse
        println("x and y are equal");
    } else {
        println("x and y are not equal");
    }
}
```

Conversion

La conversion de type permet de transformer une expression d'un type en un autre type compatible.

La conversion de type s'effectue via la syntaxe suivante :

```
(type) (z); // L'expression z est converti en (type)
```

Cette syntaxe décrit un cast explicite, mais le langage Deca permet également des conversions implicites dans certains cas :

- Conversion vers le même type.
- Conversion d'un type `int` en `float`.
- Conversion d'une classe enfant en sa classe parente (upcast).

Ces conversions implicites sont effectuées automatiquement par le compilateur sans nécessiter de cast explicite.

Transtypage

Le transtypage définit la capacité de conversion d'un type primitif à un autre.

Il est possible de convertir des entiers (type `int`) en décimaux (type `float`) et inversement. Les décimaux convertis en entier sont arrondi à l'entier inférieur.

Exemples

```
{
    float a = 3.14;
    int b = (int) (a);
    println(b); // 3

    int a = 3;
    float b = (float) (a);
}
```

```
println(b); // 3.000000e+00  
}
```

ATTENTION

La conversion d'un nombre vers un autre format peut entraîner une perte de précision.

Upcast

Un objet enfant peut être typé comme son parent.

```
class A{  
    int x = 0;  
}  
  
class B extends A{  
    int x = 1;  
}  
  
{  
    B a = new B();  
    println( ((A) (a)).x ); // Explicitation de l'upcast nécessaire pour  
    accéder à l'attribut x de A  
}
```

Downcast

Il est possible de convertir un objet parent en un de ses enfants.

```
class A{  
    int x = 0;  
}  
  
class B extends A{  
    int x = 1;  
}  
  
{
```



```
A a = new B();  
println( ((B) (a)).x );  
}
```

Erreurs Possibles

Cannot cast expression of type X to type Y

Cannot cast void type

Cannot cast expression to type X

Méthodes Assembleur

Description

La construction `asm("...")` permet d'insérer directement des instructions assembleur IMA au sein du code généré, tout en conservant la possibilité de spécifier des paramètres venant en Deca. Elle peut être utilisée pour optimiser des parties critiques du code ou pour accéder à des fonctionnalités spécifiques qui ne sont pas directement disponibles en Deca.

Syntaxe générale

```
typeRetour ident(typeParam1 param1, typeParam2 param2...) asm("
    instructions
    RTS
");
```

Accès aux paramètres

La convention de liaison utilisée par le compilateur permet d'accéder aux paramètres passés aux méthodes via la pile. Ils sont accessibles en utilisant des offsets négatifs depuis le registre de base de la pile (LB).

Le paramètre implicite `this` est toujours accessible via l'offset `-2(LB)`. Puis suivent les paramètres supplémentaires :

- `-3(LB)` pour le premier paramètre.
- `-4(LB)` pour le 2eme paramètre.
- `-5(LB)` pour le 3eme paramètre.
- `-6(LB)` pour le 4eme paramètre.
- ...

Gestion des registres

La machine abstraite IMA dispose de 16 registres à usage général pour stocker des valeurs. Comme ils sont partagés entre les méthodes, il est important de sauvegarder et de restaurer les registres utilisés dans une méthode ASM pour éviter de corrompre l'état du programme.

La convention d'utilisation des registres est la suivante :

- R0 et R1 sont des registres "scratch" et peuvent être utilisés sans être sauvegardés. Un appel à une méthode est susceptible de modifier ces registres. C'est à l'utilisateur de les sauvegarder s'il souhaite conserver leur valeur.
- R2 à R15 sont des registres "non-scratch". Chaque appelant à une méthode s'attend à ce que ces registres soient sauvegardés et restaurés par la méthode appelée. Si une méthode utilise ces registres, elle doit les sauvegarder au début de la méthode et les restaurer avant de retourner.

Exemple correct

```
void write(int n) asm("
    LOAD -3(LB), R1
    WINT
    RTS
");
```

Exemple incorrect

```
void write(int a, int b) asm("
    LOAD -3(LB), R2
    ADD -4(LB), R2 ; ! Modifie R2 sans le sauvegarder
    LOAD R2, R1
    WINT
    RTS
");
```

Dans cet exemple incorrect, le registre R2 est modifié sans être restauré, ce qui pourrait provoquer des erreurs imprévisibles dans le reste du programme.

```
void write(int a, int b) asm("
    PUSH R2 ; Sauvegarde R2
    LOAD -3(LB), R2
    ADD -4(LB), R2
    LOAD R2, R1
    WINT
    POP R2 ; Restaure R2
    RTS
");
```

RTS

Une méthode ASM doit toujours se terminer par l'instruction **RTS** (Return from Subroutine) pour indiquer la fin de la méthode et retourner au point d'appel.

Elle doit placer l'éventuelle valeur de retour dans le registre R0 avant d'appeler **RTS**.

Erreurs possibles

Comportements indéfinis

Erreurs à la compilation

Erreurs de types

X does not refer to a value/type.

Cette erreur intervient lorsqu'une variable ou un type/class est utilisé sans avoir été déclaré.

POUR CORRIGER L'ERREUR

Déclarer la variable, le type ou la classe avant de l'utiliser.

Variable/Param cannot have type void

Cette erreur intervient lorsqu'une variable ou un paramètre de méthode est déclaré avec le type `void`.

POUR CORRIGER L'ERREUR

Changer le type de la variable pour un type valide.

Variable X is already declared

Cette erreur intervient lorsqu'une variable est déclaré plusieurs fois dans la même portée.

POUR CORRIGER L'ERREUR

Supprimer les déclarations en double ou changer le nom d'une des déclarations.

Incompatible types for assignment: expected X, got Y

Cette erreur intervient lorsque le type d'une variable et sa valeur ne sont pas compatibles



POUR CORRIGER L'ERREUR

Changer le type de la variable ou la valeur.

Compared operands aren't supported

Cette erreur intervient lorsque le type d'utilisé dans une comparaison n'est pas supporté.
(Par exemple un string)



POUR CORRIGER L'ERREUR

Changer le type de l'opérande.

Erreurs de types en conditions

Condition must be of type boolean

Cette erreur intervient lorsqu'une expression dans une condition ne retourne pas un boolean.

Exemples

```
if (2) {  
  int a = 6;  
  if (a) {
```



POUR CORRIGER L'ERREUR

Modifier la condition pour avoir une condition valide.

Erreurs dans une expression booléenne

All Operands should be boolean

Cette erreur intervient lorsqu'une des deux expressions d'un opérateur logique (ET / OU logique) n'est pas un boolean.

Exemples

```
if (2 && true) {
```



POUR CORRIGER L'ERREUR

Modifier l'expression pour avoir une expression valide.

Erreurs dans une expression arithmétique/logique

Operands compared should be float or int

Cette erreur intervient lorsqu'une des deux expressions d'un opérateur arithmétique n'est pas un nombre.

Exemples

```
if (2 > true) {
```



POUR CORRIGER L'ERREUR

Modifier l'expression pour avoir une expression valide.

Both sides of a binary expression must have the same type.

Identique à l'erreur suivante.

```
print(10 / "string");
```

Cannot compare operands of different types.

Cette erreur intervient lorsqu'une des deux expressions d'un opérateur arithmétique ne sont pas du même type (entier et décimaux).

Exemples

```
if (2 > 2.0) {
```



POUR CORRIGER L'ERREUR

Modifier l'expression pour avoir une expression valide.

Convertir les entiers en décimal ou l'inverse.

Types of operands are not compatible for exact comparison: A and B

Les opérateurs `==` et `!=` ne peuvent pas être utilisé pour comparer deux types différents.

Exemples

```
if (2 == 2.0) {
```



POUR CORRIGER L'ERREUR

Convertir les décimaux en entier ou l'inverse.

The modulo operator is only supported with integers.

Cette erreur intervient lorsque l'opérateur modulo est utilisé avec un autre type qu'un entier.

Exemples

```
float b = 2.5;  
int a = 5 % b;
```

POUR CORRIGER L'ERREUR

Utiliser des entiers à la place des décimaux

Arithmetic operations are only applicable to int and float types

Cette erreur intervient lorsque les opérateur arithmétiques sont utilisés avec un autre type qu'un entier ou un décimal.

Exemples

```
print("foo" + "bar");
```

POUR CORRIGER L'ERREUR

Utiliser des entiers ou des décimaux.

Incompatible type for unary minus : expected float or int, got X

Cette erreur intervient lorsque l'opérateur `-` est utilisé avec un autre type qu'un entier ou un décimal.

Exemples

```
float b = 2.5;  
int a = 5 % b;
```



POUR CORRIGER L'ERREUR

Utiliser des entiers ou des décimaux.

Erreurs lors d'un print

Only string, int and float may be printed

Les instructions print peuvent prendre en paramètre uniquement des chaînes de caractères, des entiers, et des décimaux.

Si d'autres types sont passés en paramètre cette erreur sera levée.



POUR CORRIGER L'ERREUR

Supprimer les paramètres non valides.

Erreurs lors d'une déclaration de classe

Field cannot have type void

Un attribut d'une classe est de type `void`.

Exemples

```
class Array {  
    void buffer;  
}
```



POUR CORRIGER L'ERREUR

Changer le type de l'attribut pour un type valide.

Field X is already declared

Deux attributs d'une même classe portent le même nom.

Exemples

```
class Foo {  
    int x, x;  
}
```



POUR CORRIGER L'ERREUR

Retirer les attributs définis en double ou les renommer de manière à ce qu'ils soient uniques.

Class already exist

Deux classes portent le même nom.

Exemples

```
class Foo {  
}
```

```
class Foo {  
}
```

 POUR CORRIGER L'ERREUR

Renommer les classes de manière à ce qu'elles soient uniques.

Superclass X is not a class

La classe étendue n'est pas une classe valide.

Exemples

```
class Foo extends int {  
}
```

 POUR CORRIGER L'ERREUR

Etendre les classes uniquement avec d'autres classes et non pas des types primitifs ou non déclarés.

Field X cannot shadow a Y in the superclass

La classe enfant possède un attributs Y du même nom que la classe parent.

Exemples

```
class Bar{  
    int x = 6;  
}  
class Foo extends Bar {  
    int x = 5;  
}
```

POUR CORRIGER L'ERREUR

Utiliser la valeur définis dans la classe parent.
Changer le nom de l'attribut dans la classe enfant.

Erreurs relatives aux classes

Unable to select a field of a non-class type

Une sélection est écrite sur un type primitif.

Exemples

```
{  
    4.abs;  
}
```

POUR CORRIGER L'ERREUR

Vérifier que le type voulu est utilisé dans la sélection. Si l'objectif est de créer des attributs pour des types primitifs, créer plutôt une méthode sur un nouvel objet.

Object X does not have an attribute Y

Une sélection est écrite sur un attribut qui n'est pas déclaré.

Exemples

```
class Pair {  
    int left;  
    int right;  
}  
  
{
```

```
Pair pair = new Pair();
pair.middle = 5;
}
```

POUR CORRIGER L'ERREUR

Modifier la sélection pour accéder à un attribut existant sur le type donné. Si l'attribut souhaité existe mais sur une classe enfant, il est nécessaire de changer le type déclaré pour y instancier.

Protected field cannot be accessed outside of its class

Un attribut en visibilité `protected` est lu en dehors de la classe.

Exemples

```
class A {
    protected int b;
}

{
    A a = new A();
    a.b;
}
```

POUR CORRIGER L'ERREUR

Changer la visibilité de l'attribut. Créer un getter ou un setter.

Erreurs relatives aux méthodes

Param X is already declared

Le paramètre X est présent plusieurs fois dans la signature de la méthode.

Exemples

```
class A {  
    int foo(int a, int a){}  
}
```



POUR CORRIGER L'ERREUR

Changer le nom d'un des paramètres

Method X is overridden with different number of parameters

Une méthodes définie dans la classe enfant est déjà défini dans la classe parent mais les paramètres ne sont pas identiques.

Exemples

```
class A {  
    int foo(int a, int b){}  
}  
  
class B extends A {  
    int foo(int a){}  
}
```



POUR CORRIGER L'ERREUR

Faire correspondre les paramètres de la méthode de la classe enfant à la méthode de la classe parent.

Method X is overridden with a different return type

Une méthodes définie dans la classe enfant à un type de retour différent du type de retour de la méthode défini dans la classe parent.

Exemples

```
class A {  
    int foo(){}  
}  
  
class B extends A {  
    float foo(){}  
}
```



POUR CORRIGER L'ERREUR

Faire coïncider les types de retour.

Method X is overridden using a Y parameter, while expecting type Z

Une méthodes définie dans la classe enfant à le même nom de paramètre que la méthode défini dans la classe parent mais les types de certains paramètres ne correspondent pas.

Exemples

```
class A {  
    int foo(int a){}  
}  
  
class B extends A {  
    int foo(float a){}  
}
```



POUR CORRIGER L'ERREUR

Faire coïncider les types des paramètres.

Unable to call a method on the X primitive type

Un appel de méthode a été réalisé sur un autre type qu'une classe

Exemples

```
{  
    int.call();  
}
```



POUR CORRIGER L'ERREUR

Il n'est pas possible d'appeler une méthodes sur un types primitives.

X does not refer to any existing method

Un appel de méthode a été réalisé alors qu'aucune méthode avec ce nom n'est définie dans la classe.

Exemples

```
class A {  
    void foo() {  
    }  
}  
{  
    new A().bar();  
}
```



POUR CORRIGER L'ERREUR

Utiliser une méthode définie.

Trying to use the X **Y** as a method

Un appel de méthode a été réalisé sur un attribut de la classe.

Exemples

```
class A {  
    int foo;  
}  
{  
    new A().foo();  
}
```



POUR CORRIGER L'ERREUR

Supprimer les parenthèses pour utiliser l'attributs. Renommer la méthode ou l'attributs.

Does not match number of Method parameters of X, Y expected, but found Z

Un appel de méthode ne contient pas le bon nombre de paramètres.

Exemples

```
class A {  
    void foo(int a, int b, int c){  
    }  
}  
{  
    new A().foo(1, 2);  
}
```



POUR CORRIGER L'ERREUR

Faire coïncider les paramètres entre la signature de la méthode et l'appel à la méthode.

Parameter X does not match Method type of Y, expected A but found B

Un appel de méthode contient le bon nombre de paramètres mais ne sont pas du même type que la signature de la méthode.

Exemples

```
class A {  
    void foo(int a, int b, int c){  
    }  
}  
{  
    new A().foo(1, true, 3);  
}
```

 **POUR CORRIGER L'ERREUR**
Faire coïncider les types des paramètres entre la signature de la méthode et l'appel à la méthode.

Cannot return a void value

Une méthode contient un `return` sans valeur de retour.

Exemples

```
class A {  
    void foo(){  
        return;  
    }  
}
```

 **POUR CORRIGER L'ERREUR**
Ajouter une valeur de retour. Supprimer le `return`

Error while trying to access an attribute

L'accès à un attribut correspond à une méthode.

Exemples

```
class A {  
    int a  
}  
{  
    new A().a()  
}
```



POUR CORRIGER L'ERREUR

Supprimer l'appel à l'attribut a.

Erreur avec l'instruction *instanceof*

Left operand of the instanceof needs to be an object

L'opérande gauche de l'instruction *instanceof* doit être un objet.

Exemples

```
class A {  
}  
  
{  
    int entier = 4;  
    boolean b = entier instanceof A;  
}
```



POUR CORRIGER L'ERREUR

Assurer vous que l'opérande gauche de l'*instanceof* est un objet instancié.

Right operand of the instance of needs to be a class

Exemples

```
class A {  
}  
  
{  
    A a = new A();  
    boolean b = a instanceof int;  
}
```



POUR CORRIGER L'ERREUR

Assurer vous que l'opérande droite de l'*instanceof* est bien une classe.

Erreurs lors d'un transtypage

Cannot cast expression of type X to type Y

Un cast est effectué entre deux types non compatibles pour une assignation.

Exemples

```
class A {  
}  
  
{  
    A objet = new A();  
    int entier = (int) (objet);  
}
```



POUR CORRIGER L'ERREUR

Assurer vous que les deux types sont compatible avant d'effectuer un cast. Le transtypage n'est pas possible entre des types primitifs et des classes ou leur instances.

De même, cette erreur sera levée en cas de cast entre des objets sans lien d'héritage entre leur classes.

Cannot cast void type

Impossible de caster vers *void*, il ne s'agit pas d'un type instanciable.

Exemples

```
{
    int entier = 4;
    (void) (entier);
}
```



POUR CORRIGER L'ERREUR

Assurer vous que le type cible d'un transtypage n'est pas *void*.

Cannot cast expression to type X

Un cast est effectué sur une méthode sans valeur de retour.

Exemples

```
class A {
    void method(){}
}

{
    A a = new A();
    int b = (int)(a.method());
}
```



POUR CORRIGER L'ERREUR

Assurez-vous que la méthode possède un type de retour. Celui-ci doit être compatible pour un cast vers le type désiré.

Erreurs liés aux tableaux

Index value type must be int

Cette erreur se produit lorsque vous essayez d'utiliser une valeur qui n'est pas un entier (`int`) comme index dans un tableau.

Exemples

```
{
    int[] a = new int[10];

    a["b"] = 4;
}
```



POUR CORRIGER L'ERREUR

Remplacez l'index non entier par un entier valide.

Array type can't be void

Cette erreur se produit lorsque vous essayez de déclarer un tableau avec un type `void`, ce qui n'est pas autorisé, car `void` ne représente pas une valeur ou un type utilisable.

Exemples

```
{
    void[] a = new void[3];
}
```

```
println("test");  
}
```

POUR CORRIGER L'ERREUR

Utilisez un type valide pour le tableau, comme int, float, boolean, ou encore un type personnalisé défini par une classe.

Arrays are not supported in this version of Deca

La version de Déca que vous avez lancé n'a pas l'option qui permet d'activer les tableaux

POUR CORRIGER L'ERREUR

Lancer `décac` avec l'option `-farray`

Array size must be an integer

L'expression passée en taille d'un tableau n'est pas un entier.

Exemples

```
{  
    void[] a = new void[3.5];  
}
```

POUR CORRIGER L'ERREUR

Indiquer un entier en taille de tableau. Convertir le décimal en entier.

Cannot index non-array type

Vous utilisez l'opérateur de sélection d'index d'un tableau sur une variable qui n'est pas un tableau.

Exemples

```
{  
    int a;  
    a[0];  
}
```



POUR CORRIGER L'ERREUR

Corriger l'erreur en changeant votre syntaxe, vérifiez le nom des variables.

Erreurs à l'exécution

Arithmetic error: Attempt to divide by zero.

Une division par 0 a eu lieu, résultant en un comportement non défini.

```
print(10 / 0);
```



POUR CORRIGER L'ERREUR

Vérifier que le diviseur est différent de zéro avant de réaliser la division.

I/O error: Failed to read formatted input.

La valeur saisie au clavier par l'utilisateur ne correspond pas au type attendu par la fonction.

Memory error: Stack Overflow

La pile d'appel de fonction a dépassé sa taille maximale.



POUR CORRIGER L'ERREUR

Vérifier que toutes les boucles ou appels récursifs ont bien une condition de sortie/d'arrêt garanti.

Memory error: Heap Overflow

Le tas a dépassé sa taille maximale allouée.



POUR CORRIGER L'ERREUR

Augmenter la mémoire allouée au tas via l'option `-t` d'IMA.

Memory error: Dereferencing a null value

Une sélection d'attribut ou de méthode est opérée sur une valeur `null`.

POUR CORRIGER L'ERREUR

Retracer d'où est émise la valeur `null` (soit manuellement, soit comme valeur par défaut des attributs d'objets). Si `null` est attendu à cet endroit, ajouter une condition vérifiant la valeur n'est pas nulle avant d'y accéder.

Overflow error: Attempt to manipulate an overflow value

La valeur manipulée dépasse les limites techniques du compilateur.

POUR CORRIGER L'ERREUR

Changer la valeur.

Logical error: Missing expected return

Une méthode qui attend une valeur de retour s'est terminé sans l'instruction `return`

Exemple

```
class A {  
    int foo() {  
        print(1);  
        // missing return, expected int but got void  
    }  
}
```

```
}  
}
```

POUR CORRIGER L'ERREUR

Ajouter les `return` manquants.

Cast error: Impossible to downcast to the target class, it is not an instance of the source class

Le transtypage entre deux classes s'est révélé impossible pendant l'exécution. La class du type de retour n'hérite pas de la class source.

POUR CORRIGER L'ERREUR

Assurer vous que le transtypage est possible entre deux classes. Il est recommandé d'utiliser l'instruction 'instanceof' avant d'effectuer un cast sensible.

Assertion failed at line: x

L'assertion à la ligne x a échoué. Cela signifie que la condition booléenne a été évaluée à *false*.

POUR CORRIGER L'ERREUR

Il pourrait être intéressant de vérifier que la condition peut être vrai.

Logical error: Negative array index

Cette erreur logique se produit lorsque vous tentez de créer ou d'accéder à un tableau en utilisant un index négatif, ce qui est invalide.

POUR CORRIGER L'ERREUR

Assurez-vous que tous les indices utilisés sont des entiers compris entre 0 et la taille du tableau - 1.

Erreurs indéfinies

La spécification de Deca et de la machine abstraite IMA ne définit pas certains comportements, ce qui peut entraîner des erreurs indéfinies. Ces erreurs peuvent survenir dans les situations suivantes :

Non-respect des conventions de liaison

Les méthodes assembleur doivent accéder correctement aux paramètres passés via la pile. Elles doivent également sauvegarder et restaurer les registres non-scratch qu'elles utilisent.

Accès à des variables non initialisées

L'accès à des variables qui n'ont pas été initialisées entraîne des comportements indéfinis. Il est important de s'assurer que toutes les variables sont correctement initialisées avant leur utilisation.